

ECM2433

Ollie Mock Paper 5

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This converted mock paper has been checked so each question is answerable and answers are not given away in the question text.

Answers and marking notes are at the end of this document.

Question 1

(a) Command-line arguments: (i) explain argc and argv. (ii) Write a short C loop that prints all command-line arguments supplied to the program, one per line. (5 marks)

(b) Dynamic 2D arrays in C: (i) write `int **allocate_2d(int rows, int cols)`. (ii) Explain how to safely free this array. (6 marks)

(c) Explain why C header files usually contain include guards such as `#ifndef`, `#define`, and `#endif` when a program is split across multiple `.c` and `.h` files. (4 marks)

(d) A generic merge sort uses `int (*compare)(const void *p, const void *q)`. Explain how casting is used inside the comparison function to sort an array of doubles. (5 marks)

```
int (*compare)(const void *p, const void *q);
```

(e) Given `char buffer[50] = "Exam";`, what are `sizeof(buffer)` and `strlen(buffer)`? Explain the conceptual difference. (5 marks)

```
char buffer[50] = "Exam";
```

(Total 25 marks)

Question 2

(a) Using the C++ STL and a lambda, write code using `std::count_if` to count how many students in a

```
std::vector<Student> scored 70 or above. (6 marks)
```

```
struct Student {  
    std::string name;  
    int mark;  
};
```

(b) Write the prototype and implementation to overload `<<` for `Student` so `std::cout << student` prints Name (Mark). (5 marks)

```
struct Student {  
    std::string name;  
    int mark;  
};
```

(c) Write a C++ snippet that outputs a double `x` to exactly 3 decimal places, including trailing zeros. (5 marks)

(d) A C++ program calls a legacy C library function `void compute_stats(int *data)`. Why wrap the declaration in `extern "C"`, and how does this relate to name mangling? (5 marks)

(e) Explain the main benefit of `std::unique_ptr` over raw pointers allocated with `new`. What happens when the `unique_ptr` goes out of scope? (4 marks)

(Total 25 marks)

Question 3

(a) For the Rust enum below, write `print_status(s: &Status)` using `match` to print "Waiting", "En route via [City]", or "Arrived". (6 marks)

```
enum Status {
    Pending,
    InTransit(String),
    Delivered,
}
```

(b) In Rust, `Rc<T>` is reference-counted shared ownership. (i) Explain `Box<T>` versus `Rc<T>`. (ii) If `Rc::clone(&my_rc)` is called, what exactly is duplicated? (5 marks)

(c) Using `iter`, `filter`, `map`, and `collect`, write one Rust expression that takes `Vec<i32>`, filters out negative numbers, adds 10 to the remaining numbers, and collects into `Vec<i32>`. (5 marks)

(d) Rust uses `?` for concise error propagation. In a function returning `Result<String, std::io::Error>`, explain what happens when `File::open("data.txt")?` encounters an error versus when it succeeds. (5 marks)

(e) Explain why the Rust code below is rejected by the compiler. (4 marks)

```
let mut text = String::from("Hello");
let r1 = &mut text;
let r2 = &mut text;
println!("{}", r1, r2);
```

(Total 25 marks)

Question 4

(a) Null pointers versus Option: explain how Rust's `Option<T>` forces the programmer to handle missing data at compile time and prevents accidental null pointer dereferences. (6 marks)

(b) Concurrency and data races: (i) In C++, what is the risk if a programmer forgets to lock a `std::mutex` before modifying a shared map? (ii) In Rust, how does `Arc<Mutex<HashMap<...>>>` prevent modification without locking? (7 marks)

(c) Compare dynamic dispatch in C++ and Rust. (i) Explain virtual functions and base class pointers in C++. (ii) Explain traits and `Box<dyn Trait>` in Rust. (6 marks)

(d) Manual versus automatic iteration: give two distinct safety benefits of Rust's `for` `x` in `&arr` style over C manual indexing with `for (int i = 0; i < N; i++)`. (6 marks)

(Total 25 marks)

Answers and marking notes

Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (5 marks): Command-line arguments: (i) explain argc and argv. (ii) Write a short C loop that prints all command-line arguments supplied to the program, one per line.

Answer: argc is the count of argument strings and argv is the array of those strings. argv[0] is normally the program name. To print user-supplied arguments only, start at index 1.

```
for (int i = 1; i < argc; i++) {
    printf("%s\n", argv[i]);
}
```

(b) Question (6 marks): Dynamic 2D arrays in C: (i) write int **allocate_2d(int rows, int cols). (ii) Explain how to safely free this array.

Answer: Allocate an array of row pointers, then allocate each row. To free it, free each row first, then the row-pointer array. On allocation failure, free anything already allocated.

```
int **allocate_2d(int rows, int cols) {
    int **data = malloc((size_t)rows * sizeof *data);
    if (data == NULL) {
        return NULL;
    }
    for (int r = 0; r < rows; r++) {
        data[r] = malloc((size_t)cols * sizeof *data[r]);
        if (data[r] == NULL) {
            for (int i = 0; i < r; i++) {
                free(data[i]);
            }
            free(data);
            return NULL;
        }
    }
    return data;
}

void free_2d(int **data, int rows) {
    for (int r = 0; r < rows; r++) {
        free(data[r]);
    }
    free(data);
}
```

(c) Question (4 marks): Explain why C header files usually contain include guards such as #ifndef, #define, and #endif when a program is split across multiple.c and.h files.

Answer: Include guards prevent the same header contents being processed more than once in a translation unit. Without them, repeated includes can cause redefinition errors for structs, typedefs, function declarations in some contexts, and other declarations.

(d) Question (5 marks): A generic merge sort uses int (*compare)(const void *p, const void *q). Explain how casting is used inside the comparison function to sort an array of doubles.

Code from question:

```
int (*compare)(const void *p, const void *q);
```

Answer: The generic sorter passes pointers as const void *. The comparator casts those pointers back to const double *, dereferences them, and compares the double values.

```
int compare_double(const void *p, const void *q) {
    const double *a = p;
    const double *b = q;
    if (*a < *b) {
        return -1;
    }
    if (*a > *b) {
        return 1;
    }
    return 0;
}
```

(e) Question (5 marks): Given char buffer[50] = "Exam";, what are sizeof(buffer) and strlen(buffer)? Explain the conceptual difference.

Code from question:

```
char buffer[50] = "Exam";
```

Answer: sizeof(buffer) is 50 because it is the storage size of the whole char array. strlen(buffer) is 4 because it counts characters before the null terminator. strlen does not include the terminator or unused array capacity.

Question 2

(a) Question (6 marks): Using the C++ STL and a lambda, write code using `std::count_if` to count how many students in a `std::vector<Student>` scored 70 or above.

Code from question:

```
struct Student {
    std::string name;
    int mark;
};
```

Answer: The lambda should return true for marks greater than or equal to 70.

```
int passes = std::count_if(students.begin(), students.end(),
    [](const Student& student) {
        return student.mark >= 70;
    });
```

(b) Question (5 marks): Write the prototype and implementation to overload `<<` for `Student` so `std::cout << student` prints Name (Mark).

Code from question:

```
struct Student {
    std::string name;
    int mark;
};
```

Answer: The operator should return `std::ostream&` so output can be chained. The `Student` parameter should usually be `const Student&`.

```
std::ostream& operator<<(std::ostream& os, const Student& student) {
    os << student.name << " (" << student.mark << ")";
    return os;
}
```

(c) Question (5 marks): Write a C++ snippet that outputs a double `x` to exactly 3 decimal places, including trailing zeros.

Answer: Use fixed plus `setprecision(3)`. `fixed` makes the precision count digits after the decimal point.

```
#include <iomanip>
std::cout << std::fixed << std::setprecision(3) << x << "\n";
```

(d) Question (5 marks): A C++ program calls a legacy C library function `void compute_stats(int *data)`. Why wrap the declaration in `extern "C"`, and how does this relate to name mangling?

Answer: C++ name mangling changes symbol names to support features such as overloading. The C object file exports a C-style symbol. `extern "C"` tells C++ to use C linkage for the declaration so the linker can match the names.

```
extern "C" {
    void compute_stats(int *data);
}
```

(e) Question (4 marks): Explain the main benefit of `std::unique_ptr` over raw pointers allocated with `new`. What happens when the `unique_ptr` goes out of scope?

Answer: `unique_ptr` gives RAII ownership of a heap object. It automatically deletes the object when the `unique_ptr` goes out of scope, preventing leaks on normal returns and exceptions. It cannot be copied, which protects unique ownership.

Question 3

(a) Question (6 marks): For the Rust enum below, write `print_status(s: &Status)` using `match` to print "Waiting", "En route via [City]", or "Arrived".

Code from question:

```
enum Status {
    Pending,
    InTransit(String),
    Delivered,
}
```

Answer: Match on the borrowed enum and use the city binding in the `InTransit` case.

```
fn print_status(s: &Status) {
```

```
    Answer: match s { Status::Pending => println!("Waiting"), Status::InTransit(city) => println!("En route via {city}"), Status::Delivered
=> println!("Arrived"),
    }
}
```

(b) Question (5 marks): In Rust, `Rc<T>` is reference-counted shared ownership. (i) Explain `Box<T>` versus `Rc<T>`. (ii) If `Rc::clone(&my_rc)` is called, what exactly is duplicated?

Answer: `Box<T>` gives one owner of heap data. `Rc<T>` gives multiple owners in one thread by keeping a reference count. `Rc::clone` duplicates the `Rc` pointer/handle and increments the reference count; it does not clone the heap data.

(c) Question (5 marks): Using `iter`, `filter`, `map`, and `collect`, write one Rust expression that takes `Vec<i32>`, filters out negative numbers, adds 10 to the remaining numbers, and collects into `Vec<i32>`.

Answer: With `iter`, `filter` needs to account for references. `map` then copies out the integer and adds 10.

```
let result: Vec<i32> = values
```

```
Answer: .iter().filter(|&n| **n >= 0).map(|&n| *n + 10).collect();
```

(d) Question (5 marks): Rust uses `?` for concise error propagation. In a function returning `Result<String, std::io::Error>`, explain what happens when `File::open("data.txt")?` encounters an error versus when it succeeds.

Answer: If `File::open` returns `Err(e)`, `?` returns that error immediately from the current function. If it returns `Ok(file)`, `?` unwraps the file value and execution continues. The function's return type must be compatible with the error.

(e) Question (4 marks): Explain why the Rust code below is rejected by the compiler.

Code from question:

```
let mut text = String::from("Hello");
let r1 = &mut text;
let r2 = &mut text;
```

Answer: Rust allows only one active mutable reference to a value at a time. `r1` is still active because it is used in `println`, so creating `r2` as another mutable borrow of `text` is rejected. This rule prevents data races and aliasing mutation bugs.

Question 4

(a) Question (6 marks): Null pointers versus `Option`: explain how Rust's `Option<T>` forces the programmer to handle missing data at compile time and prevents accidental null pointer dereferences.

Answer: `Option<T>` is either `Some(value)` or `None`. Code cannot use the inner `T` directly without matching, `if let`, `unwrap`, or another explicit handling method. This makes the missing case visible in the type system instead of allowing a hidden null pointer to be dereferenced accidentally.

(b) Question (7 marks): Concurrency and data races: (i) In C++, what is the risk if a programmer forgets to lock a `std::mutex` before modifying a shared map? (ii) In Rust, how does `Arc<Mutex<HashMap<...>>>` prevent modification without locking?

Answer: Forgetting to lock in C++ can allow concurrent unsynchronised access, causing a data race and undefined behaviour. In Rust, `Arc` gives shared ownership of the `Mutex`, not direct mutable access to the `HashMap`. To mutate the map, code must call `lock()` and obtain a `MutexGuard`. The guard controls access and unlocks when dropped.

(c) Question (6 marks): Compare dynamic dispatch in C++ and Rust. (i) Explain virtual functions and base class pointers in C++. (ii) Explain traits and `Box<dyn Trait>` in Rust.

Answer: C++ uses virtual member functions so a call through a base pointer/reference dispatches to the derived override at runtime. Rust uses traits to define shared behaviour, and `Box<dyn Trait>` stores a heap value behind a trait object so calls can dispatch dynamically through the trait.

(d) Question (6 marks): Manual versus automatic iteration: give two distinct safety benefits of Rust's `for` in `&arr` style over C manual indexing with `for (int i = 0; i < N; i++)`.

Answer: Rust iteration over `&arr` avoids manually writing indexes, so there is less chance of off-by-one or out-of-bounds access. The iterator only yields valid references to existing elements. It also avoids mismatching `N` with the actual array length and works with borrowing rules that prevent invalid mutation while iterating.